

Verifier-Centered Auto-Research Agents for Lie-Group Multibody Integrator Development: A Versioned Case Study

Jingquan Wang
Simulation-Based Engineering Laboratory
University of Wisconsin–Madison

June 9, 2026

Abstract

Large-language-model agents can edit code, launch experiments, and summarize logs, but those capabilities are not sufficient for numerical-method research. In Lie-group multibody dynamics, a candidate integrator can produce an attractive convergence plot while failing to preserve velocity-level constraints, relying on an unreported endpoint projection, comparing against an unfair reference policy, or exceeding what its proof actually establishes. This paper proposes a verifier-centered architecture for building domain-specific auto-research agents for constrained Lie-group integrator development. The central design principle is that the agent must co-design four objects: the method specification, the verifier sandbox, the proof ledger, and the numerical example ladder. The resulting workflow represents each candidate method as an explicit one-step-map object, runs bounded validators before full artifact generators, separates theorem inputs from diagnostic evidence, and records claim states as accepted, diagnostic, open, or forbidden. We ground the architecture in a versioned research tree that evolved from v001 to v048 for lower-pair multibody integrator work. The current case study centers on a Gauss6/FullVA path with a conditional sixth-order method claim, smooth observed position/velocity orders of approximately 7.161/7.066, four ASME mechanism examples in the evidence system, and explicit nonpromotion boundaries for full source-paper temporal finite-element replacement, source-policy external superiority, sparse runtime superiority, and sharp-friction coarse-regime behavior. The contribution is not a generic “research agent” recipe. It is a concrete methodology for using agents in a research domain where proof obligations, numerical examples, and paper claims must remain synchronized over many iterations and many contributors.

Keywords: auto-research agents; multibody dynamics; Lie-group integration; constrained differential-algebraic equations; numerical verification; proof ledgers; reproducibility.

1 Introduction

Multibody simulation is useful only when the time integrator respects the geometric and algebraic structure of the mechanism being simulated. In rigid and flexible multibody systems, finite rotations must remain on the relevant Lie group, bilateral joints must satisfy position-, velocity-, and acceleration-level constraints, and reaction forces or friction laws may change the engineering conclusion drawn from the simulation. These requirements make integrator research different from ordinary software development. A script can run successfully, a plot can look plausible, and a regression test can pass, while the method is still not a defensible integrator.

This distinction matters for laboratories that develop simulation methods over many projects. In one part of the lab, researchers may work on Lie-group Runge–Kutta or collocation schemes. In another, they may work on lower-pair constraints, friction/contact laws, sparse Newton solvers, public benchmark mechanisms, or reproducible paper packages. These activities interact through a shared evidence space. A change in the stage residual can invalidate a proof ledger; a change in the reference policy can invalidate a work-precision comparison; a mechanism that closes position constraints may still be only a coverage row rather than

an accepted asymptotic dynamic-order row. An auto-research agent for this setting is valuable only if it coordinates these dependencies instead of merely generating code.

This is also why the problem is practically important inside a lab rather than only philosophically interesting. Multiple researchers may be attacking the same integrator family from different directions: one person improving a Gauss-collocation residual, another building an ASME mechanism mapping, another checking public-code baselines, and another preparing a manuscript. Without a shared verifier and memory system, those threads can drift into mutually inconsistent claims. The agent proposed here is a coordination mechanism for that research ecology: it keeps each person’s artifact useful to the others by recording exactly what the artifact proves, what it only diagnoses, and what it does not yet justify.

The motivating case in this paper is a versioned research pipeline for Lie-group, lower-pair multibody integrator development. The pipeline currently contains a Gauss6/FullVA path, a temporal finite-element (TFE) comparison target, ASME-style mechanism examples, proof and claim ledgers, and a cross-paper same-test benchmark scaffold. The artifact state is deliberately bounded. The accepted local method claim is a conditional sixth-order Gauss6/FullVA claim with smooth observed position/velocity orders about 7.161/7.066. Four ASME examples enter the method evidence system; single- and double-pendulum rows support dynamic-order evidence, while four-link and slider-crank rows support mechanism coverage and closed-loop consistency unless a stronger residual-to-error theorem or true dynamic-order campaign is closed. The pipeline does not claim complete source-paper TFE residual reproduction, full source-policy external superiority, or sparse runtime superiority.

This paper asks: how should an auto-research agent be built for this kind of integrator research? The answer proposed here is verifier-centered. The agent does not treat experiments as a flat task list. It maintains an explicit method object, chooses a sandbox level before execution, keeps proof obligations in a separate ledger, selects numerical examples to expose specific failure modes, and writes paper claims only through a claim-state machine. In this architecture, autonomy is not measured by how many commands the agent can run without a human. It is measured by how reliably the agent prevents unverified claims from crossing the boundary into the paper.

The paper makes four contributions:

1. It formulates a domain-specific architecture for auto-research agents in Lie-group multibody integrator development.
2. It defines the core research objects—MethodSpec, ProblemSpec, EvidenceSpec, and ClaimSpec—that let an agent preserve method identity and paper-claim boundaries.
3. It introduces verifier and proof sandbox patterns that separate read-only validators, bounded audits, full artifact generation, and external source-policy campaigns.
4. It grounds the framework in a v001–v048 versioned case study, showing how each phase of the project added a verifier because a previous phase exposed a new failure mode.

2 Background and Motivation

2.1 Constrained multibody integration

Constrained multibody dynamics is commonly formulated as a differential- algebraic system with kinematic constraints, velocity constraints, acceleration constraints, and Lagrange multipliers. Classical references on stiff ordinary differential equations and DAEs emphasize that the numerical method must be matched to the index and stability structure of the problem [3, 4]. For mechanical systems, the problem is compounded by rotations. Lie-group integration methods address the fact that rigid-body orientations are not Euclidean vectors [5, 6]. Constrained Lie-group time integration has therefore become a recurring theme in multibody dynamics [7, 8].

The practical difficulty is that no single plot captures all relevant properties. Position error, velocity error, constraint drift, multiplier consistency, Newton convergence, runtime, and reference-policy fairness are different pieces of evidence. A method that performs well in a smooth single-body test may fail in a lower-pair mechanism. A method that satisfies endpoint position constraints may still hide a velocity inconsistency.

A method that has an exact sparse pattern may still lose to a dense automatic differentiation backend in wall-clock time.

2.2 Temporal finite elements and comparison targets

Time finite element methods provide another way to construct high-order time-integration schemes for structural and multibody dynamics [15, 16, 17, 18, 19]. The motivating source-paper family includes higher-order integration of index-3 DAEs with friction using time finite elements on Lie groups [1]. In the local pipeline, the source-paper TFE formulation and the implemented Gauss6/FullVA path are deliberately distinguished. The accepted local method claim is not that the complete source-paper residual has been reimplemented. Rather, the local TFE target appears as a formal-order comparison object: a paper-style $m = 3$ Gauss-Lobatto TFE target with expected order five, contrasted against the conditional sixth-order Gauss6/FullVA path.

This distinction is a representative example of why an agent needs a claim ledger. A generic agent may see TFE-related diagnostics, convergence plots, and source-policy files and compress them into a single statement that “the TFE method was reproduced.” That statement would be wrong. The artifact boundary is narrower: some formula mappings and diagnostics exist, but `full_tfe_stage_replacement=false` remains an open stronger reproduction target.

2.3 External baselines and source-policy fairness

External comparison introduces a second class of failure. A local method may look superior if it uses a different time window, a different reference solution, a different error metric, or a different work metric. The relevant baseline family in the case study includes SBEL/Negrut mechanisms and rotation-representation work, including exponential-map, absolute-coordinate, half-implicit, and velocity-partitioning/Lie-group baselines [21, 22, 23, 24, 25]. The v048 scaffold therefore treats external comparison as a source-policy object. Until same-test campaigns are run with matched parameters, drivers, reference policies, error metrics, and work metrics, the claim `external_superiority_claim=false` remains active.

3 Problem Statement and Design Requirements

The task is to build an agent that helps researchers search the space of integrators, repairs, examples, proofs, and paper claims without corrupting the evidence boundary. We model the agent as a controller over a research state

$$\mathcal{S} = (\mathcal{M}, \mathcal{P}, \mathcal{E}, \mathcal{C}, \mathcal{L}),$$

where $\mathcal{M}$ is the method specification, $\mathcal{P}$ is the problem or benchmark specification, $\mathcal{E}$ is the evidence set, $\mathcal{C}$ is the claim set, and $\mathcal{L}$ is the ledger containing proof, validation, version, and paper-package status. The agent proposes actions, but every claim-changing action must pass through a verifier.

3.1 Failure modes

The case study suggests seven recurring failure modes:

1. **Method drift.** The agent modifies rows, variables, tolerances, or endpoint policies and then compares the result as if it were the same integrator.
2. **Projection masking.** A post-step projection repairs a constraint residual but changes the one-step map.
3. **Proof/numerics confusion.** Observed slopes or fixed-tolerance logs are promoted into theorem inputs.
4. **Coverage/order confusion.** A mechanism coverage row is written as if it were an accepted asymptotic dynamic-order row.

5. **Reference-policy drift.** External baselines use different time windows, references, or work metrics.
6. **Negative-result erasure.** Failed candidates are not recorded, causing repeated exploration of the same invalid repair.
7. **Paper-claim overreach.** A mechanically buildable paper package is described as submission-ready despite open source-policy or quality gates.

3.2 Requirements

A useful agent for this domain must satisfy the following requirements:

- R1: Method identity.** The one-step map must be explicitly represented, including state space, stage variables, residual rows, endpoint policy, and solver policy.
- R2: Deterministic problem specifications.** Benchmarks must pin masses, inertias, joints, drivers, friction parameters, step sizes, reference policies, and metrics.
- R3: Sandboxed execution.** Read-only validators, targeted audits, full generators, and external campaigns must be separated.
- R4: Proof separation.** Theorem inputs must be kept separate from diagnostic numerical evidence.
- R5: Example ladder.** Numerical examples must be selected to expose specific failure modes before broader claims are made.
- R6: Negative evidence.** Failed repairs must record residuals, ranks, spans, runtime, and claim effects.
- R7: Claim-state control.** Claims must be stateful and demotable: accepted, diagnostic, open, opt-in, or forbidden.

4 Agent Architecture

4.1 Research objects

The proposed architecture is organized around four objects. Table 1 summarizes their roles.

Table 1: Core objects maintained by the domain-specific auto-research agent.

Object	Contents	Why it matters
MethodSpec	State space, stage variables, residual rows, endpoint map, solver policy	Prevents the agent from changing the method identity while preserving the name.
ProblemSpec	Mechanism, parameters, drivers, friction law, time window, step sizes, reference policy	Makes examples reproducible and comparable.
EvidenceSpec	CSV rows, JSON summaries, plots, validator outputs, proof certificates	Defines what counts as evidence for a gate.
ClaimSpec	Claim text, status, required validators, nonpromotion rules	Prevents diagnostic evidence from becoming paper claims.

The agent loop is then:

read ledgers → select open gate → propose bounded hypothesis
→ choose sandbox → run verifier → update evidence/claims.

The human chooses the research direction, but the agent enforces the evidence boundary before any claim moves. This is the opposite of a code-only agent. The primary output is not a patch; it is a controlled transition in the research state.

4.2 Skills and memory

The agent is intentionally assembled from narrow skills rather than from a single generic code-editing loop. In the local artifact system, the explicitly named process guard is the **research-pipeline** skill: it forces the agent to read current artifacts before relying on memory and prevents open gates from being marked complete. Around that guard, the implementation uses several conceptual roles. The *numerical-method* role knows the objects that define a Lie-group integrator: charts, retractions, stage variables, residual rows, endpoint maps, tolerances, and constraint levels. The *example* role maps a requested mechanism into masses, inertias, joints, drivers, reference policy, step sizes, and error metrics. The *paper* role turns accepted claims into manuscript text and demotes prose that exceeds the artifact state. The deterministic *review* role is implemented by the local review artifacts `cmame_submission_review_agent.py`, `validate_cmame_review_agent.py`, and `CMAME_REVIEW_AGENT_REPORT.md/json`.

The most important design choice is that memory is stored in artifacts, not in hidden conversation state. Table 2 gives the memory layout used by the agent. This makes the workflow reproducible across sessions and across people in the lab: a new agent run can reconstruct the current claim boundary from files, rather than relying on what a previous chat happened to remember.

Table 2: Memory objects used by the agent.

Memory	Artifact	Role in the agent
Contract memory	<code>CURRENT_PIPELINE_CONTRACT.md</code> and current status files	Defines the accepted method, allowed paper-facing claim, open gates, and forbidden overclaims.
Version memory	<code>version_ledger.csv</code> , version reports, and progression plots	Records why each version exists, including failed attempts and blocker-driven turns.
Proof memory	<code>ORDER_PROOF_LEDGER.md</code> , proof contracts, and traceability audits	Separates theorem assumptions, discharged lemmas, diagnostic probes, and retained conditions.
Numerical memory	CSV rows, result matrices, plots, and summary JSON files	Stores errors, orders, residuals, ranks, constraints, timings, and reference policies.
Source-policy memory	cross-paper benchmark specifications and public-code runner summaries	Prevents local diagnostics from becoming external superiority claims before same-test campaigns close.
Paper memory	manuscript text, figure manifests, references, and submission-readiness notes	Ensures prose is synchronized with accepted evidence rather than with stale ambitions.

4.3 Step-level verification loop

Each agent action is run through a fixed verification loop. The loop is simple enough to be used repeatedly, but strict enough to catch the common failure modes in integrator research:

1. **Load authority.** Read the current contract, status, version ledger, proof ledger, result matrix, and source-policy specification.
2. **Pin identity.** Freeze the MethodSpec and ProblemSpec for the proposed action, including residual rows, endpoint closure, solver tolerance, reference policy, and metrics.
3. **Choose sandbox.** Classify the action as read-only validation, bounded probe, full artifact generation, or source-policy campaign.
4. **Run the smallest verifier.** Prefer read-only claim audits and bounded probes before full regeneration; require explicit opt-in for heavy public same-test campaigns.
5. **Recompute evidence.** Parse the generated CSV/JSON artifacts, recompute errors and observed orders, and compare them to the expected gate.

6. **Update claim state.** Promote, demote, or leave claims unchanged according to the gate result; never promote a diagnostic row into a theorem input by prose alone.
7. **Write durable memory.** Update ledgers, reports, and paper text only after the verifier state and claim state agree.

Algorithm 1 Verifier-centered research step

Require: Current ledgers $\mathcal{L}$, proposed action a , method/problem state $(\mathcal{M}, \mathcal{P})$

Ensure: Updated evidence $\mathcal{E}'$ and claim state $\mathcal{C}'$, or a recorded nonpromotion

- 1: Read the pipeline contract, version ledger, proof ledger, result matrix, and source-policy specification
 - 2: Freeze the MethodSpec and ProblemSpec for a
 - 3: Select the smallest sandbox level $s \in \{L0, L1, L2, L3\}$
 - 4: Run the verifier allowed by s and parse generated CSV/JSON/PDF artifacts
 - 5: **if** the verifier changes method identity or reference policy unexpectedly **then**
 - 6: demote the result to diagnostic and record the mismatch
 - 7: **else if** the verifier closes all requirements for a claim **then**
 - 8: promote the claim only through the corresponding ClaimSpec transition
 - 9: **else**
 - 10: keep the claim open and record the missing gate
 - 11: **end if**
 - 12: Run reviewer-agent checks against the updated manuscript/prose boundary
 - 13: **if** reviewer flags overclaim or stale evidence **then**
 - 14: revise prose, table row, or claim state; do not promote without a verifier rerun
 - 15: **end if**
 - 16: Write durable memory: reports, ledgers, result matrix, and manuscript text
-

This loop is deliberately conservative. For example, if a closed-loop four-link residual is small but there is no accepted residual-to-error theorem, the agent writes “closed-loop consistency evidence” rather than “accepted dynamic sixth-order evidence.” If a sparse Jacobian pattern is exact but the dense automatic-differentiation backend is faster in wall-clock measurements, the agent writes “sparse structure verified” rather than “sparse speed superiority.”

4.4 Reviewer agent

The workflow also includes a reviewer agent. This is not a second authoring agent that rewrites the paper freely. It is a read-only critic whose job is to compare the manuscript against the current artifacts and return findings that force another update pass. The reviewer agent is useful because the authoring agent is biased toward coherence: after it has written a clean narrative, it may miss that a phrase quietly exceeds the ledger. The reviewer agent is biased toward contradiction. In the current package, this lane is represented by `cmame_submission_review_agent.py`, validated by `validate_cmame_review_agent.py`, and summarized in `CMAME_REVIEW_AGENT_REPORT.md/json`. The report may accept the narrowed claim while still keeping global source-policy and reproducibility submission readiness false.

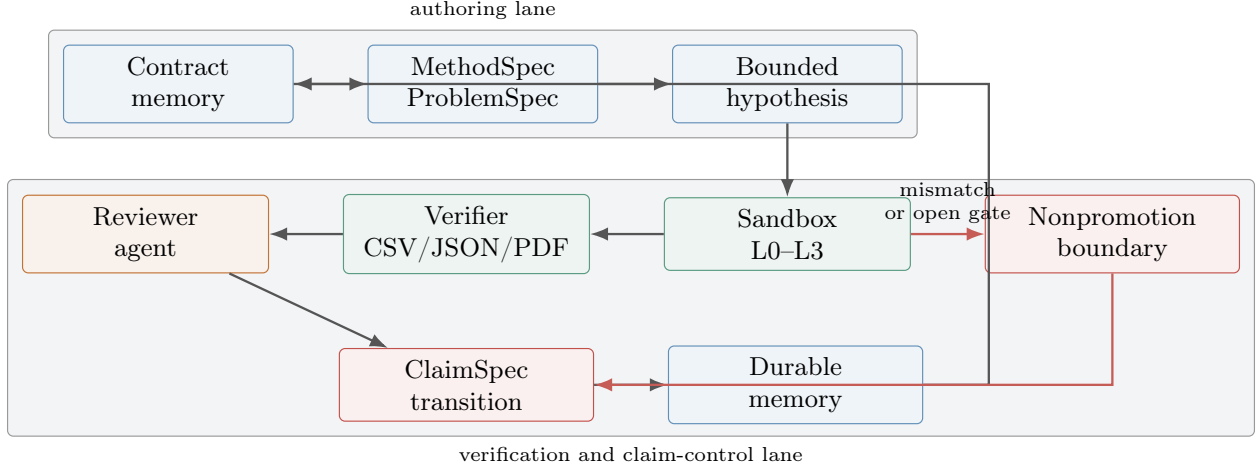


Figure 1: The method architecture becomes a verifier target. The agent must keep the residual, proof boundary, validators, and artifact layer synchronized instead of treating the implementation as an opaque script.

Table 3: Reviewer-agent checklist used before accepting a manuscript update.

Review question	Evidence checked	Revision trigger
Does the evolution story match the ledger?	<code>version_ledger.csv</code> , version reports, and current contract	A version is described as a success when the ledger records it as a blocker, diagnostic, or partial result.
Are ASME rows correctly classified?	ASME CSV rows, result matrix, and current pipeline contract	Four-link or slider-crank coverage is written as accepted dynamic-order proof.
Are reference policies explicit?	Example CSVs, cross-paper benchmark spec, and result matrix	A local self-reference, analytic reference, and public-source reference are mixed in one comparison.
Are proof assumptions still visible?	Proof ledger, proof contract gate, and traceability audit	P1/P2/P3/P4/P6 retained conditions, P7, or η_h solver-policy boundaries disappear from prose.
Are external claims over-promoted?	v048 source-policy summary and cross-paper benchmark spec	Bounded public-code pilots are written as source-policy external superiority.
Is the paper status over-stated?	current contract, proof audit, and package-status files	A mechanically buildable PDF is called globally submission-ready.

The update rule is: authoring pass, reviewer pass, revision pass, and then build verification. A reviewer finding can demote a claim, require a new artifact citation, split a table row into accepted and diagnostic lanes, or force an explicit limitation paragraph. A reviewer finding cannot by itself promote a result. Promotion still requires the relevant verifier sandbox and claim-state transition.

4.5 Claim state machine

Each claim is assigned one of five states:

Hypothesis. A candidate method, repair, or comparison idea that has not yet passed a gate.

Diagnostic. Evidence that is useful for understanding the system but not sufficient for a paper claim.

Accepted. A claim whose required validators and ledgers agree.

Open. A known blocker remains active.

Forbidden. The wording conflicts with the current gate state and must be removed or demoted.

The important feature is reversibility. A claim can move backward when a validator exposes stale prose, a mismatched reference policy, or a proof obligation that was previously hidden.

5 Verifier Sandbox

The verifier sandbox constrains what the agent can run and what the resulting output is allowed to mean. Four levels are used:

Table 4: Verifier sandbox levels.

Level	Sandbox	Allowed question	Forbidden by default
L0	Read-only validators	Do current claims match current artifacts?	Regenerating results or changing status.
L1	Target audit	Can one bounded hypothesis falsify a local gap?	Promoting claims or rewriting ledgers.
L2	Full generator	After method-code changes, do version-wide artifacts regenerate?	Using stale ledgers after regeneration.
L3	External source-policy campaign	Do public same-test rows close under explicit policy?	Running expensive or policy-sensitive campaigns without opt-in.

This structure matters because different questions have different blast radii. A paper-editing question should usually be answered by a read-only validator, not by rerunning a full historical generator. A source-policy external campaign can be expensive, and its results affect paper-level claims; it should require explicit human opt-in. Conversely, after method code is changed, a full artifact generator may be necessary because old summaries are no longer authoritative.

At each level, the verifier has a different definition of success. A useful agent must keep these definitions separate, because a pass at one level does not imply a pass at another level.

Table 5: Step-level verifier outputs and nonclaims.

Step	Verifier output	Claim it can support	Claim it cannot support
Contract audit	Accepted method name, order claim, open caveats, and stale tokens	Paper prose matches current artifacts	Numerical correctness or theorem closure
Method audit	Residual-row count, endpoint map, solver policy, and stage variables	The implementation is the intended candidate	Error order or external superiority
Example audit	Mechanism parameters, drivers, step sizes, references, and metrics	The benchmark is reproducible	Method superiority across papers
Convergence audit	Error table and observed slopes from at least three step sizes	Local asymptotic evidence under the stated reference	A theorem unless proof conditions are closed
Proof audit	Closed lemmas, retained assumptions, row certificates, and theorem interface	Conditional theorem statement with explicit assumptions	Empirical validation of open assumptions
Source-policy audit	Matched public parameters, references, work metrics, and runner status	Same-test readiness or completed same-test rows	External superiority if the campaign is not run

In the case study, the sandbox prevents three concrete mistakes. First, endpoint projection evidence is kept separate from raw residual and KKT closure evidence. Second, full-TFE replacement scans remain diagnostic until they close the appropriate residual-substitution and order gates. Third, v048 same-test scaffolds do not promote external-superiority claims until the source-policy campaign is actually run.

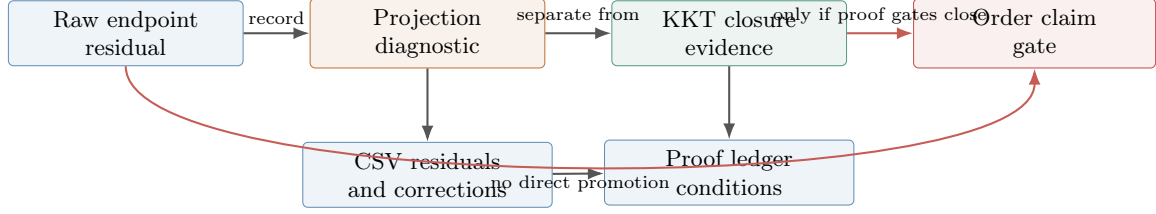


Figure 2: Endpoint closure is sandboxed as its own evidence stream. Projection, raw residual, and KKT closure are recorded separately so that constraint repair does not silently become an order claim.

6 Proof Sandbox

The proof sandbox specifies what may enter the theorem. For the Gauss6/FullVA path, the current theorem-facing route is a conditional direct residual-bridge route rather than a fully closed primitive Taylor-subterm route. The proof ledger therefore records both accepted interfaces and retained conditions.

The theorem is decomposed around a one-step map. Let ϕ_h denote the exact flow in a smooth reduced chart, Ψ_h^G the ideal three-stage Gauss map, and Ψ_h^A the implemented Gauss6/FullVA map after stage residual, endpoint closure, and inexact Newton effects. The proof target is not to show that every piece of the implementation is symbolically identical to a source-paper TFE residual. It is to show, under stated regularity and solver assumptions, that the implemented map differs from the exact flow by a local defect of order h^7 :

$$\|\Psi_h^A(y) - \phi_h(y)\| \leq C_G h^7 + C_A h^7 + C_E h^7 + C_N \eta_h.$$

If the Newton tube tolerance satisfies $\eta_h \leq c_\eta h^7$, then

$$\|\Psi_h^A(y) - \phi_h(y)\| \leq C_{\text{loc}} h^7, \quad C_{\text{loc}} = C_G + C_A + C_E + C_N c_\eta.$$

One-step stability, $\|\Psi_h(y) - \Psi_h(\bar{y})\| \leq (1 + C_s h) \|y - \bar{y}\|$, then transfers the local defect to a global sixth-order bound over $[0, T]$:

$$\max_{0 \leq nh \leq T} \|y_n - y(t_n)\| \leq C_{\text{loc}} \frac{\exp(C_s T) - 1}{C_s} h^6.$$

This is why the paper-facing claim is sixth order even when some smooth observed slopes are higher. The observed slopes are evidence for the implemented path; the theorem is bounded by the closed proof obligations.

Table 6: Proof obligations tracked by the agent.

ID	Obligation	Reason
P1	Smooth reduced chart and FullVA lift	Gauss order must transfer through constrained coordinates.
P2	Compact-tube branch, stage Jacobian, stability, and endpoint-closure interfaces	Newton and endpoint closure must define a local one-step map on the accepted branch.
P3	Runtime formula and automatic-differentiation oracle for the 132-row residual	The implemented residual layout must be the residual being analyzed.
P4	Non-dynamic $O(h^7)$ residual certificate and endpoint perturbation interface	The 96 non-dynamic rows and endpoint effects must not lower the local defect order.
P5	Newton–Euler dynamic rows discharged by direct substitution	Dynamic force-balance rows cannot be hand-waved.
P6	Solver residual $\eta_h \leq ch^7$	Fixed production tolerances are not theorem inputs.
P7	Residual-to-error theorem for surrogate rows	Mechanism residual checks are not automatically order proofs.

The residual proof is then split by rows and by route. The current implementation has a 132-row FullVA residual. The proof audit records a 96-row non-dynamic certificate and a 36-row dynamic Newton–Euler direct-substitution check. These numbers matter because they prevent the agent from writing “all rows are symbolically proved” when only a subset is proved by a particular lane.

Table 7: Residual-proof decomposition used by the agent.

Block	Rows or interface	Current status	How the agent uses it
Non-dynamic residual certificate	96 residual rows	Closed for the direct $O(h^7)$ certificate lane	Supports the implemented residual-defect block, not external comparison.
Dynamic Newton–Euler rows	36 rows	Closed by direct substitution on the current route	Prevents dynamic force-balance rows from being treated as informal diagnostics.
P-interface assumptions	P1–P6	Retained theorem conditions plus partial certificates	Defines the conditional theorem statement and prevents unconditional wording.
Residual-to-error bridge	P7	Open	Keeps closed-loop four-link and slider-crank residual checks as coverage/consistency evidence.
Solver route	tolerance $\eta_h \leq c_\eta h^7$	Theorem condition retained; finite probes are diagnostics	Blocks fixed production tolerances from becoming proof-level closure.

Algorithm 2 Proof-sandbox audit for the Gauss6/FullVA route

Require: 132-row residual implementation, proof ledger, Newton tolerance policy, ASME evidence rows

Ensure: Conditional theorem boundary and nonpromotion list

- 1: Verify that the runtime formula and AD oracle identify the same 132 residual rows
 - 2: Check the 96 non-dynamic row certificate for $O(h^7)$ defect on the accepted lift
 - 3: Check the 36 Newton–Euler rows by direct substitution on the current route
 - 4: Retain P1/P2 compact-branch, smooth-lift, endpoint-closure, and stability assumptions
 - 5: Retain P6 unless $\eta_h^{\text{tube}} \leq c_\eta h^7$ is proved as a theorem-level solver policy
 - 6: **if** all local-defect blocks close under retained assumptions **then**
 - 7: allow the conditional local defect $O(h^7)$ and global sixth-order statement
 - 8: **end if**
 - 9: **if** P7 residual-to-error obligations are not all closed **then**
 - 10: classify four-link/slider-crank residual rows as coverage and consistency only
 - 11: **end if**
 - 12: Forbid unconditional theorem, source-policy superiority, and residual-to-error promotion
-

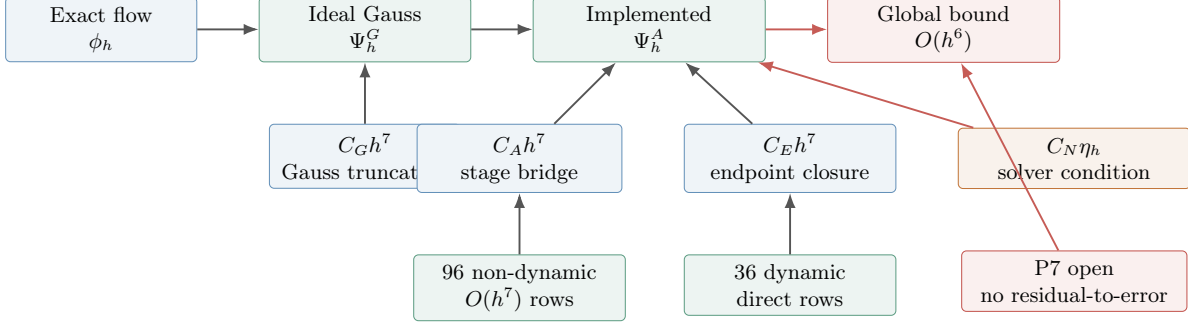


Figure 3: Proof sandbox decomposition. The conditional theorem route is built from local-defect blocks and row certificates, while P7 prevents closed-loop residual diagnostics from becoming trajectory-order proof.

The P7 boundary is especially important for the four-link and slider-crank examples. The proof contract records seven obligations that must close before closed-loop reaction residuals can be used as trajectory-error or order proof:

Table 8: Open residual-to-error obligations for P7.

ID	Blocking obligation
P7.1	Dynamic residual identity.
P7.2	Residual consistency rate $O(h^7)$.
P7.3	Closed-loop DAE stability or inf-sup bound.
P7.4	Calibrated non-floor-limited residual-to-error estimator.
P7.5	Reference-floor exclusion.
P7.6	Coarse-first accepted dynamic-order campaign.
P7.7	Manuscript theorem and proof.

The ledger separates an accepted lane from a diagnostic lane. Accepted evidence includes direct residual bridge checks, automatic-differentiation-expanded row identities, and Newton–Euler direct-substitution certificates for the current conditional route. Diagnostic evidence includes primitive Taylor fragments, fixed-tolerance logs, residual-only closed-loop rows, and external source-policy rows. This separation is essential because numerical evidence can support a theorem but cannot substitute for a theorem condition.

7 Numerical Example Ladder

The agent chooses numerical examples by the failure mode they expose. The ladder used in the case study is:

1. SO(3) kinematics: right-action, chart, and log-map checks.
2. Smooth mechanics: invariants and high-order smooth behavior.
3. Constrained DAEs: position, velocity, and acceleration closure.
4. Friction regimes: regularization width, pre-asymptotic order, and cost-to-recover order.
5. ASME mechanisms: single pendulum, double pendulum, four-link, and slider-crank lower-pair coverage.
6. External same-test baselines: matched parameters, references, errors, and work metrics.

The ladder prevents premature promotion. The agent should not jump from a toy kinematics plot to a paper claim about lower-pair multibody dynamics. Nor should it use a closed-loop residual row as an external dynamic-order row until the missing residual-to-error or same-test campaign is closed.

8 Four ASME Example Protocols

The four ASME-style examples are not decorative demonstrations. They are verifiers with different jobs. The agent therefore stores, for each example, the mechanism definition, the reference policy, the error norm, the order formula, and the allowed claim state. This is the part of the workflow that prevents a visually plausible mechanism animation from being overinterpreted as a method-order theorem.

8.1 Metric and reference policies

The examples do not share one universal error metric. The agent instead records the metric with the artifact that produced it. This is a critical detail because the v047 closed-loop kinematic/reaction rows and the later common-reference result matrix answer different questions.

Table 9: Metric policies for ASME-related artifacts.

Artifact lane	Reference and steps	Metric	Allowed use
Single-pendulum analytic method row	Analytic driven-angle reference; local method sweep $h = \{0.20, 0.10, 0.05\}$ over $T = 0.2$	Position/velocity, orientation-log, and angular-velocity errors from the method CSV	Accepted local method-side dynamic-order evidence.
Double-pendulum local method row	Nested local Gauss6/FullVA reference $h = 0.005$; tested $h = \{0.04, 0.02, 0.01\}$ over $T = 0.2$	Orientation and angular-velocity errors plus constraint residual diagnostics	Accepted local method-side dynamic-order evidence under the local reference policy.
Closed-loop kinematic/reaction rows	Local kinematic FullVA reference $h = 0.001$; tested $h = \{0.02, 0.01, 0.005\}$ over $T = 0.2$	Trajectory L_∞ consistency, constraint residuals, multiplier reconstruction, Newton–Euler residuals	Four-link/slider-crank mechanism coverage and consistency only. The near-floor slopes are not dynamic-order claims.
Common-reference result matrix	Common diagnostic sweep $h = \{0.1, 0.05, 0.025\}$ with reference $h = 0.0125$	Recorded <code>final_12</code> or <code>final_linf</code> ; each cell reports observed order / finest-step error	Bounded common-reference diagnostic. It is not source-policy external superiority.

Within any one lane, the observed order is computed only from errors produced under that same lane’s reference and metric. With three nested step sizes h_0, h_1, h_2 satisfying $h_{i+1} = h_i/2$, the agent records the log–log slope

$$\hat{p} = \frac{\log(e_{h_0}/e_{h_2})}{\log(h_0/h_2)},$$

with pairwise slopes and reference-floor warnings retained as diagnostics. If the finest errors are at roundoff or reference-floor level, the row is not promoted to an asymptotic order claim even if the formal slope is high. Constraint residuals and Newton–Euler residuals are stored separately; they are consistency diagnostics unless the residual-to-error theorem closes.

Algorithm 3 Constructing one numerical evidence row

Require: Example m , method candidate, lane policy ℓ , step list H , reference policy r , metric d

Ensure: Evidence row with errors, observed order, diagnostics, and claim scope

- 1: Load mechanism parameters, joints, drivers, initial state, horizon, and tolerances for m
 - 2: Generate or load the reference trajectory/state according to r
 - 3: **for** each $h \in H$ **do**
 - 4: Run the method under the lane policy ℓ
 - 5: Compute the lane-specific error $e_h = d(y_h, y_{\text{ref}})$
 - 6: Record constraint residuals, Newton diagnostics, and reaction residuals separately
 - 7: **end for**
 - 8: Compute $\hat{p} = \log(e_{h_0}/e_{h_2})/\log(h_0/h_2)$ and pairwise slopes
 - 9: **if** reference floor, nonmonotonic errors, or open P7/source-policy gate is detected **then**
 - 10: mark the row as diagnostic or coverage-only
 - 11: **else**
 - 12: mark the row as accepted for the specific ClaimSpec it satisfies
 - 13: **end if**
 - 14: Write CSV/JSON/table entries with the reference, metric, finest error, order, and claim scope
-

Table 10: Protocol summary for the four ASME examples and their evidence lanes.

Example	Mechanism and reference	Compared quantities	Allowed interpretation
Single pendulum	Driven ASME single pendulum; analytic driven-angle reference; $T = 0.2$, method-row $h = \{0.20, 0.10, 0.05\}$ and common-matrix $h = \{0.1, 0.05, 0.025\}$.	COM position/velocity, orientation log error, angular velocity, drive residual diagnostics, and final-error matrix row.	Accepted local method-side dynamic-order evidence; external rows remain diagnostics unless source-policy closes.
Double pendulum	Two-revolute ASME double pendulum mapped to the FullVA side method; nested local reference $h = 0.005$ for method row; common-matrix reference $h = 0.0125$.	Orientation and angular velocity errors plus position-, velocity-, and acceleration-level constraint residuals.	Accepted local method-side dynamic-order evidence under the local reference policy; external rA comparison remains pending.
Four-link local coverage	Driven closed-loop mechanism with 18 generalized coordinates and 18 constraints; local kinematic FullVA reference $h = 0.001$; $h = \{0.02, 0.01, 0.005\}$.	Trajectory L_∞ consistency, constraint residuals, and reconstructed Newton–Euler reaction residuals.	Mechanism coverage and closed-loop consistency, not accepted dynamic-order proof.
Four-link common-reference diagnostic	Same mechanism in the paper result matrix; reference $h = 0.0125$; $h = \{0.1, 0.05, 0.025\}$.	<code>final_linf</code> position/velocity/acceleration errors reported as order / finest-step error.	Internal common-reference diagnostic; not a source-policy or residual-to-error promotion.
Slider-crank local coverage	Closed-loop slider-crank with DP1, DP2, coordinate-difference, and distance constraints; local kinematic FullVA reference $h = 0.001$; $h = \{0.02, 0.01, 0.005\}$.	Trajectory L_∞ consistency, multiplier reconstruction, and translational/rotational dynamic residuals.	Mechanism coverage and closed-loop consistency, not accepted dynamic-order proof.
Slider-crank common-reference diagnostic	Same mechanism in the paper result matrix; reference $h = 0.0125$; $h = \{0.1, 0.05, 0.025\}$.	<code>final_linf</code> position/velocity/acceleration errors reported as order / finest-step error.	Internal common-reference diagnostic; not external superiority and not residual-to-error closure.

Table 11: Numerical evidence rows for the four ASME examples in the common-reference matrix.

Example	Reference, metric, and sweep	Position/velocity order and finest error	Acceleration row	Claim scope
Single pendulum	Analytic driven-angle reference; final_12 ; $h = \{0.1, 0.05, 0.025\}$.	Position: 6.013 / 3.323×10^{-14} . Velocity: 5.801 / 1.155×10^{-12} .	Not a promoted matrix claim.	Consistent with the accepted local dynamic-order row; not external superiority.
Double pendulum	Local Gauss6/FullVA final-state reference at $h = 0.0125$; final_linf ; same sweep.	Position: 6.011 / 3.100×10^{-13} . Velocity: 6.010 / 2.346×10^{-13} .	Not a promoted matrix claim.	Common-reference diagnostic plus accepted local dynamic-order row under its own reference policy.
Four-link	Closed-loop endpoint reference at $h = 0.0125$; final_linf ; same sweep.	Position: 5.955 / 2.289×10^{-12} . Velocity: 6.085 / 1.093×10^{-11} .	0.995 / 8.80×10^{-1} .	Coverage and consistency only; P7 blocks dynamic-order promotion.
Slider-crank	Closed-loop endpoint reference at $h = 0.0125$; final_linf ; same sweep.	Position: 6.164 / 8.583×10^{-12} . Velocity: 7.341 / 9.189×10^{-12} .	1.054 / 1.04×10^{-1} .	Coverage and consistency only; P7 blocks dynamic-order promotion.

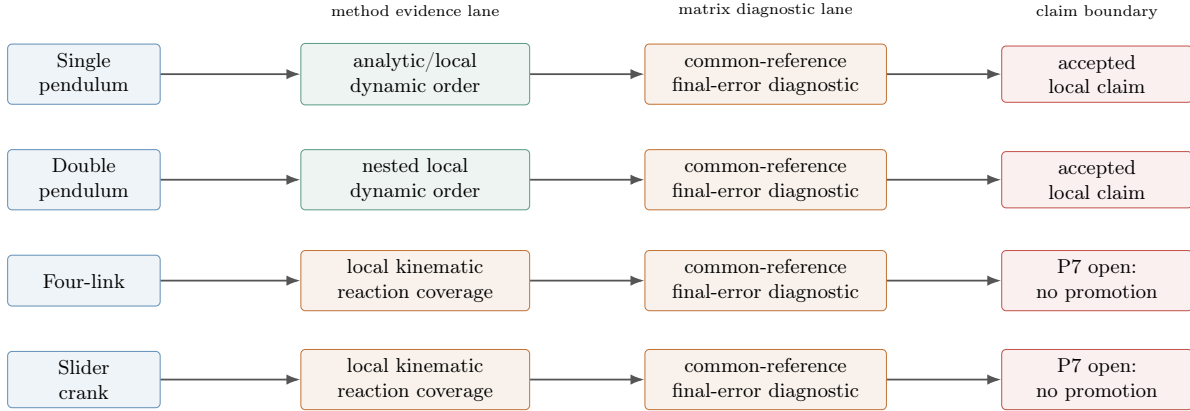


Figure 4: ASME evidence lanes. The examples share a result matrix but do not share the same claim status: single and double support local dynamic-order evidence, while four-link and slider-crank remain coverage/consistency until P7 closes.

8.2 Single pendulum

The single-pendulum example is the cleanest dynamic-order verifier because the drive provides an analytic reference. The public mechanism uses a length 4 m body, square side length 0.05 m, density 7800 kg/m³, gravity 9.81 m/s², and a DP1 drive of the form

$$\cos\left(\frac{\pi}{2} + \frac{\pi}{4} \cos(2t)\right).$$

The local Gauss6/FullVA row solves right-Lie orientation increments together with center-of-mass position, velocity, acceleration, angular velocity, angular acceleration, pivot reaction, axis multipliers, and drive multiplier. The residual includes pivot position, pivot velocity, pivot acceleration, revolute-axis DP1 rows, and acceleration-level drive closure.

The reference is the analytic driven angle and its derivatives, sampled on the same grid as the numerical result. The primary comparisons are position, velocity, orientation, and angular velocity. In the current

accepted row, the three-step sweep reports observed orders approximately 6.073 for position, 6.033 for velocity, 6.055 for orientation, and 6.024 for angular velocity. This is why the row supports the local sixth-order method claim.

8.3 Double pendulum

The double-pendulum row tests whether the FullVA mechanism survives a genuine two-joint coupled system. The ASME mechanism has two bodies of lengths 4 m and 2 m, square side length 0.05 m, density 7800 kg/m³, gravity 9.81 m/s², and two revolute joints. The local mapping uses the v029-style double-revolute Gauss6/FullVA side method. Public zero initial angular speeds are regularized by opposite 10⁻¹² perturbations to avoid a singular local convention; this regularization is recorded as part of the ProblemSpec and is not hidden.

The reference is a nested local Gauss6/FullVA trajectory at $h = 0.005$, while the tested step sizes are $h = \{0.04, 0.02, 0.01\}$ over $T = 0.2$. The accepted comparisons are orientation and angular velocity errors, with constraint residuals retained as consistency checks. The current row reports observed orders approximately 6.101 for orientation and 6.089 for angular velocity. This is accepted method-side dynamic-order evidence, but it is not yet a same-test public rA superiority claim because the public-source reference and work-policy campaign is a separate v048 gate.

8.4 Four-link

The four-link row is deliberately tagged differently. The mechanism has three moving bodies with masses 2, 1, 1 and inertias (4, 2, 0), (12.4, 0.01, 0), and (4.54, 0.01, 0) under the local convention. The driver is $\cos(\pi t + \pi/2)$. The local closed-loop kinematic FullVA system uses 18 generalized coordinates and 18 constraints, including six DP1 rows and twelve coordinate-difference rows. The recorded minimum singular value of the kinematic constraint Jacobian is about 7.92×10^{-2} , with condition number about 1.17×10^2 .

The v047 local coverage reference is a kinematic FullVA trajectory at $h = 0.001$, while the coverage sweep uses $h = \{0.02, 0.01, 0.005\}$ over $T = 0.2$. That sweep is a nested sampling and reproducibility check: the recorded trajectory errors are near the reference floor and the formal slopes are not interpreted as dynamic order. A separate multiplier reconstruction checks

$$\Phi_q^\top \lambda = \begin{bmatrix} F - Ma \\ -(J\alpha + \omega \times J\omega) \end{bmatrix},$$

and records Newton–Euler residuals around 10⁻¹³. This supports closed-loop consistency, not an external-order claim.

The paper result matrix has a separate common-reference diagnostic lane for the same mechanism. There, the local Gauss6/FullVA row uses $h = \{0.1, 0.05, 0.025\}$, reference $h = 0.0125$, and `final_linf` errors. That lane reports position/velocity slopes about 5.955 and 6.085 with finest position/velocity errors about 2.289×10^{-12} and 1.093×10^{-11} . Because this lane is a common-reference diagnostic and because P7 remains open, those slopes are not promoted into closed-loop dynamic-order proof.

8.5 Slider-crank

The slider-crank example stresses mixed lower-pair constraints. The public mechanism distinguishes the 2021 and 2022 parameter policies; the local 2021-style row uses masses 0.12, 0.5, and 2 with inertias (10⁻⁴, 10⁻⁵, 10⁻⁴), (4 × 10⁻³, 4 × 10⁻⁴, 4 × 10⁻³), and (10⁻⁴, 10⁻⁴, 10⁻⁴). The driver is $\cos(-2\pi t + \pi/2)$. The local constraint inventory includes seven DP1 rows, four DP2 rows, six coordinate-difference rows, and one distance row, with a recorded minimum singular value about 7.25×10^{-2} and condition number about 43.6.

As in the four-link case, the v047 local coverage reference is a kinematic FullVA trajectory at $h = 0.001$ with $h = \{0.02, 0.01, 0.005\}$ over $T = 0.2$. Those local kinematic rows record near-floor trajectory and constraint consistency; they are not the source of a dynamic-order claim. The reconstructed reaction-dynamics residuals are near 6×10^{-15} across the sweep.

The result matrix again contains a separate common-reference diagnostic lane with $h = \{0.1, 0.05, 0.025\}$, reference $h = 0.0125$, and `final_linf` errors. That lane reports position/velocity slopes about 6.164 and

7.341 with finest position/velocity errors about 8.583×10^{-12} and 9.189×10^{-12} . The acceleration slope is not used as a method-order claim because the acceleration column is dominated by reconstruction and floor behavior. The correct paper wording is therefore “slider-crank closed-loop coverage plus common-reference diagnostics,” not “accepted slider-crank dynamic sixth-order proof” and not “public baseline superiority.”

9 Versioned Case Study: v001 to v048

The version history is the strongest evidence that the agent must be domain-specific. The versions are not a changelog; they are a record of blockers. Each phase added a verifier because the previous phase exposed a new way an integrator could fail.

The baseline before v001 was a conventional research workflow: implement a candidate formula, run a small number of scripts, inspect convergence plots, and then summarize the result in prose. That workflow is adequate for many ordinary software experiments, but it breaks down for this project because the same word “method” can hide different residuals, endpoint policies, reference solutions, and proof assumptions. The shift from the baseline to v001 was therefore not merely the creation of a first script. It was the first attempt to make method identity auditable. The shift from v001 to the current v048 endpoint was then driven by successive discoveries that a new piece of the research state had to be made explicit.

Table 12: Blocker-driven evolution of the research pipeline.

Versions	Phase	Why the project evolved
v001–v005	Lie-group basics	Establish $SO(3)$ order, right-action conventions, and smooth Gauss mechanics before adding constraints.
v006–v013	Constrained DAE closure	Endpoint position, velocity, and acceleration consistency became explicit verifier targets.
v014–v022	Friction regimes	Sharp regularized friction exposed delayed asymptotic behavior and cost-to-recover-order questions.
v023–v029	FullVA emergence	Absolute-coordinate lower-pair attempts exposed endpoint velocity/acceleration consistency failures.
v030–v045	Solver scaling	Dense Jacobians and dense solves became bottlenecks, so sparse pattern and coloring audits became separate claims.
v046–v048	Claim gates and external policy	ASME examples, proof ledgers, paper packages, and same-test scaffolds became first-class objects.

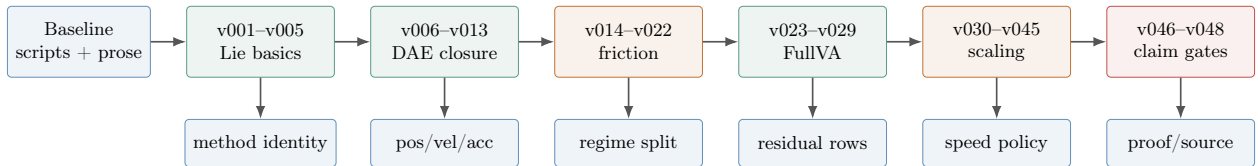


Figure 5: Version evolution as a sequence of forced verifier additions. The pipeline did not evolve because later versions were cosmetically cleaner; it evolved because each phase exposed a new ambiguity that the agent had to make auditable.

Table 13: Fine-grained evolution from baseline to the current v048 endpoint.

Stage	What was learned	Why the next stage was needed	Verifier added
Baseline	Public mechanisms, source-paper TFE ideas, and local scripts existed as separate objects.	The workflow could not distinguish formula comparison, source-code reproduction, and paper claim readiness.	Claim-state boundary and durable artifact memory.
v001	First SO(3) benchmark suite gave a controlled Lie-group kinematics baseline.	Correct SO(3) motion was necessary but not enough; action convention and chart choice could silently change the method.	Right-action and log-map orientation checks.
v002–v005	CF4/RKMK4 repairs, SBEL rA reproduction, Yoshida midpoint, and Gauss-Lie4 smooth mechanics clarified fourth-order behavior.	Smooth unconstrained order did not test DAE closure, lower-pair constraints, or endpoint velocity/acceleration consistency.	Smooth mechanics convergence and invariant checks.
v006–v008	Fixed-pivot and absolute-coordinate DAE experiments showed that position closure alone was insufficient.	Endpoint constraints had to include position, velocity, and acceleration levels, otherwise projections could mask method drift.	Position/velocity/acceleration closure validators.
v009–v013	Quaternion/JAX/Gauss6 variants achieved smooth high order but exposed sharp-friction order reduction.	The agent needed to separate smooth theorem evidence from nonsmooth or regularized-friction regimes.	Smooth-vs-sharp friction sweeps and cost-to-recover-order diagnostics.
v014–v022	Brown–McPhee and revolute benchmark stages studied friction smoothness, adaptivity, and cost.	The next blocker was no longer scalar accuracy; it was lower-pair absolute-coordinate residual design.	Friction-width ledgers and benchmark-specific diagnostics.
v023–v029	Full absolute revolute DAE attempts evolved toward Off-axis FullVA and double-revolute FullVA.	Endpoint velocity and acceleration consistency became the central method identity issue.	FullVA residual-row audits and double-revolute dynamic-order rows.
v030–v045	Sparse CSR Newton, matrix-free attempts, lagging, colored JVP/VJP, symbolic pattern, and pattern cache studied scaling.	Exact sparsity did not imply run-time superiority; speed claims needed their own evidence policy.	Sparse-pattern certificates and dense-vs-sparse timing caveats.
v046–v047	Four ASME examples, proof contracts, paper package, and claim audits were integrated.	The paper could now build mechanically, but source-policy and global submission-readiness claims were still open.	ASME result matrix, proof traceability audit, paper-claim audit.
v048	Same-test external benchmark scaffold and public-code pilot rows were added.	External superiority still required matched public parameters, references, work metrics, and completed campaigns.	Cross-paper source-policy spec and opt-in external campaign gate.

Table 14: Compact version ladder, v001–v012.

Version	Research turn	Verifier or claim-boundary effect
v001	First reproducible SO(3) benchmark suite.	Created benchmark memory before high-order claims.
v002	Corrected right-action CF4/RKMK4 fourth-order kinematics.	Made action convention and log-map order checks explicit.
v003	Reproduced SBEL/Negrut rA code path.	Added a source-code reproducibility anchor, not a high-order claim.
v004	Added Yoshida-composed Lie midpoint mechanics.	Recorded that negative substeps are poor for friction/contact.
v005	Added two-stage Gauss-Lie4 smooth mechanics.	Established smooth unconstrained order and invariant checks.
v006	Moved to reduced fixed-pivot DAE.	Separated exact reconstruction from full absolute-coordinate DAE.
v007	Implemented absolute-coordinate Gauss DAE.	Exposed the need for position, velocity, and acceleration consistency.
v008	Internalized endpoint constraints in the Newton solve.	Reduced reliance on endpoint projection.
v009	Added smooth and sharp regularized friction.	Split smooth-order evidence from sharp-friction order reduction.
v010	Replaced finite-difference Jacobians with JAX AD.	Made nonlinear-solve backend evidence a separate lane.
v011	Implemented quaternion transport operators.	Bridged local SO(3) residuals to paper-style S3 transport.
v012	Embedded quaternion endpoint DAE residual.	Verified quaternion residual identity before increasing order.

Table 15: Compact version ladder, v013–v024.

Version	Research turn	Verifier or claim-boundary effect
v013	Added Gauss6 endpoint collocation.	Produced sixth-order smooth evidence while preserving sharp-friction caveats.
v014	Swept friction smoothness.	Turned “Gauss6 is worth it” into a regime-dependent claim.
v015	Added adaptive Gauss6/Gauss4 controller.	Quantified near-nonsmooth cost/accuracy tradeoffs.
v016	Added reduced Lie-trapezoidal baseline.	Ruled out a favorable low-order baseline as the main winner.
v017	Added reduced Lie-BDF2 baseline.	Kept robustness evidence separate from accuracy leadership.
v018	Tested Lobatto endpoint collocation.	Prevented simple TFE-style endpoint-node substitution from being overclaimed.
v019	Added reaction-dependent friction.	Connected multiplier/reaction loads to the residual.
v020	Added adaptive lambda-friction controller.	Quantified sharp lambda-friction recovery.
v021	Added Brown–McPhee-style friction.	Moved from surrogate friction toward paper-like continuous friction.
v022	Added revolute Brown–McPhee benchmark.	Made a paper-like one-DOF revolute friction verifier.
v023	Built full absolute-coordinate revolute DAE.	Exposed endpoint velocity and formulation fragility.
v024	Tried direct Lobatto full revolute DAE.	Recorded failure of simple Gauss-to-Lobatto node replacement.

Table 16: Compact version ladder, v025–v036.

Version	Research turn	Verifier or claim-boundary effect
v025	Added endpoint projection repair.	Separated practical constraint repair from unchanged one-step-map claims.
v026	Added stage pivot-velocity residual.	Made velocity closure a residual-row target.
v027	Added stage pivot-acceleration residual.	Made acceleration closure a residual-row target.
v028	Added off-axis FullVA residual.	Checked axis velocity/acceleration consistency under off-axis torque.
v029	Extended to double-revolute interbody system.	Showed FullVA generalizes and exposed sparse/block Newton needs.
v030	Measured double-revolute Jacobian sparsity and CSR timing.	Turned solver scaling into a separate evidence lane.
v031	Integrated CSR sparse solves into Newton.	Showed sparse solves preserve trajectory but still depend on dense AD materialization.
v032	Tested JVP-GMRES matrix-free Newton.	Recorded matrix-free failure without preconditioning.
v033	Tested lagged sparse Newton.	Recorded that reduced Jacobian refreshes did not materially improve runtime.
v034	Tested colored-JVP sparse assembly.	Verified sparse AD correctness but rejected it as small-system speed winner.
v035	Batched colored JVP seeds.	Improved sparse AD runtime while retaining exactness checks.
v036	Generated block-symbolic sparse pattern.	Removed dense warm-up as the only pattern source.

Table 17: Compact version ladder, v037–v048.

Version	Research turn	Verifier or claim-boundary effect
v037	Pruned symbolic pattern with JVP dry-runs.	Recovered exact observed sparsity without dense Jacobian discovery.
v038	Reused cached JVP-pruned pattern.	Added cache/reuse policy for nearby regimes.
v039	Extended to triple-revolute topology.	Tested sparse AD beyond the two-body case.
v040	Generated block pattern for triple revolute.	Made larger-topology pattern acquisition more production-like.
v041	Added cache refresh/union policy.	Recorded stale-cache detection and refresh rules.
v042	Added skew-axis triple-revolute lower pairs.	Tested non-coplanar lower-pair geometry.
v043	Added skew prismatic lower pair.	Added non-revolute coverage while preserving sparse-speed caveat.
v044	Added double-prismatic interbody chain.	Exposed high color count and dense-backend competitiveness.
v045	Added row-colored VJP sparse AD.	Reduced seed count but retained dense-runtime caveat.
v046	Validated upstream ASME rA examples.	Created four-example public/reproducibility anchor without a Gauss6 method claim.
v047	Built cylindrical-chain Gauss6/FullVA pipeline and claim gates.	Integrated 132-row residual, proof audits, ASME evidence, smooth-order evidence, caveats, and paper-package validators.
v048	Built same-test external benchmark harness.	Added public-code pilots and source-policy scaffolding while keeping external superiority false.

This evolution explains why the current agent records negative and partial results instead of deleting them. v004 was useful because negative substeps were poor for friction/contact. v013 was useful because the same Gauss6 path showed smooth order near six but sharp-friction pre-asymptotic order around the low-order regime. v030–v045 were useful because they showed that a correct sparse pattern did not automatically beat a dense automatic differentiation implementation. v048 is useful even though it does not close external superiority, because it turns the public comparison into a structured source-policy campaign rather than a prose-level aspiration.

Table 18: Evidence-state matrix used by the agent.

Evidence lane		Single pendulum	Double pendulum	Four-link	Slider-crank
Local dynamic-order method row		Accepted; analytic/reference-policy order evidence	Accepted; nested local reference order evidence	Not used as local dynamic-order proof	Not used as local dynamic-order proof
Local kinematic/reaction coverage		Reaction reconstruction diagnostic	Constraint residual diagnostic	Accepted coverage and reaction consistency	Accepted coverage and reaction consistency
Common-reference result matrix		Internal order/error diagnostic	Internal order/error diagnostic	Internal final-error diagnostic only	Internal final-error diagnostic only
External source-policy claim		Open; no superiority claim	Open; no superiority claim	Open; no superiority claim	Open; no superiority claim
Proof promotion boundary		Conditional sixth-order method claim	Conditional sixth-order method claim	Blocked by P7 residual-to-error obligations	Blocked by P7 residual-to-error obligations

The evolution has two methodological implications. First, a version should be created when the research question changes, not when a script is cosmetically edited. Second, negative results are useful artifacts. A

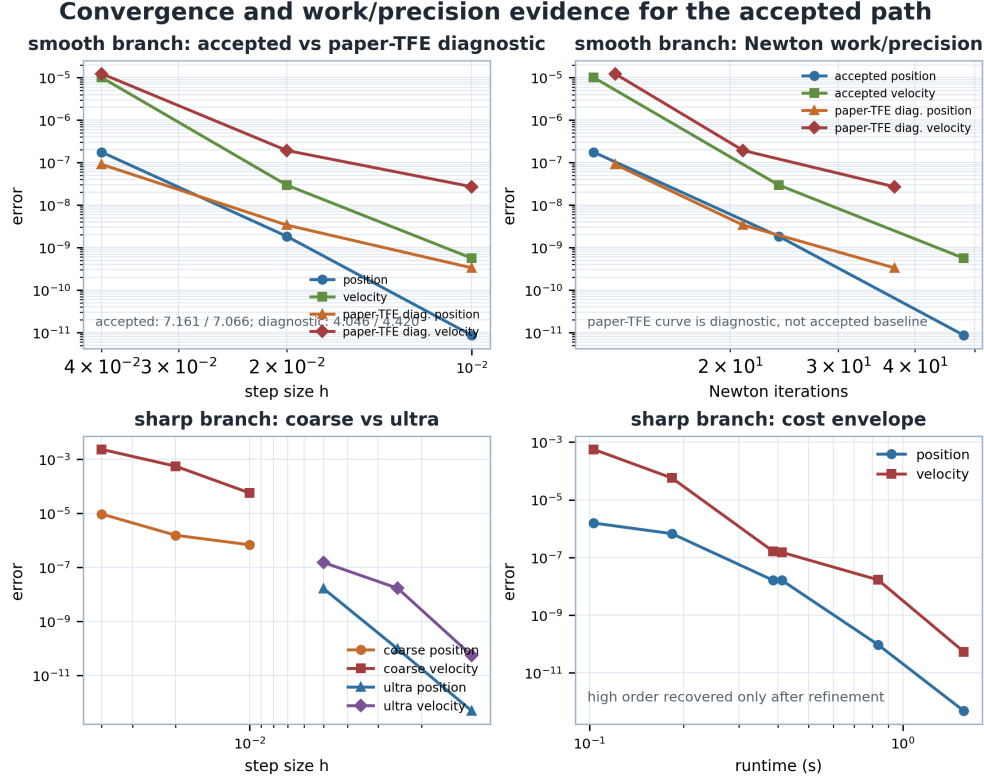


Figure 6: Smooth convergence evidence for the current local Gauss6/FullVA path. The observed slopes support the method evidence, while the theorem claim remains bounded by the proof ledger.

failed TFE replacement candidate that records rank, residual, and missing span prevents the next agent run from rediscovering the same failure.

10 Current Evidence Boundary

10.1 Accepted local method evidence

The current accepted local method object is the Gauss6/FullVA path. It has three Gauss stages and 132 FullVA residual rows. The smooth cylindrical-chain h-sweep records observed position/velocity orders about 7.161/7.066. The paper-facing method claim is conditional sixth order, not seventh order, and not full source-paper residual reproduction.

10.2 ASME mechanism evidence

The four ASME-style mechanisms enter the current method evidence system: single pendulum, double pendulum, four-link, and slider-crank. The current boundary distinguishes dynamic-order evidence from mechanism coverage. Single- and double-pendulum rows support accepted method-side dynamic-order evidence. Four-link and slider-crank rows support closed-loop kinematic FullVA coverage and reaction-dynamics consistency; they are not promoted into accepted asymptotic dynamic-order proof without a stronger theorem or campaign.

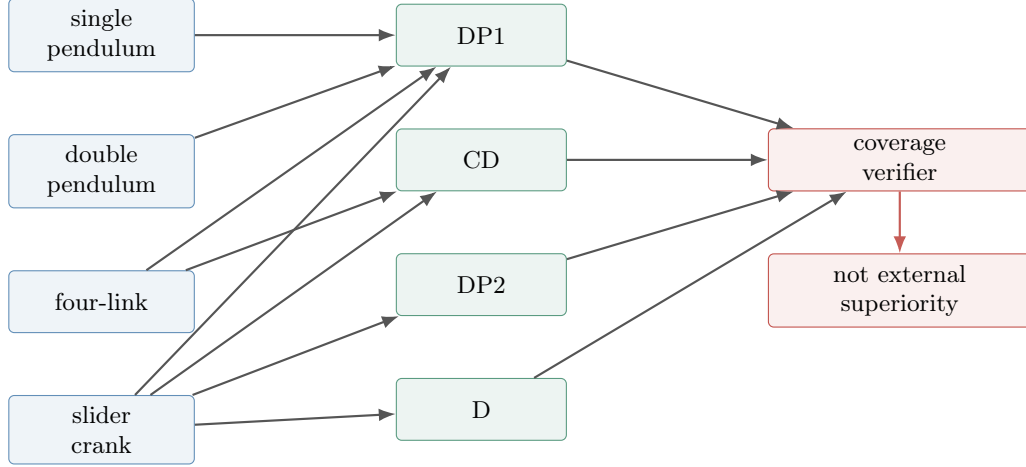


Figure 7: The lower-pair graph bridge connects mechanism examples to constraint families. It is a coverage and consistency verifier, not by itself an external-superiority claim.

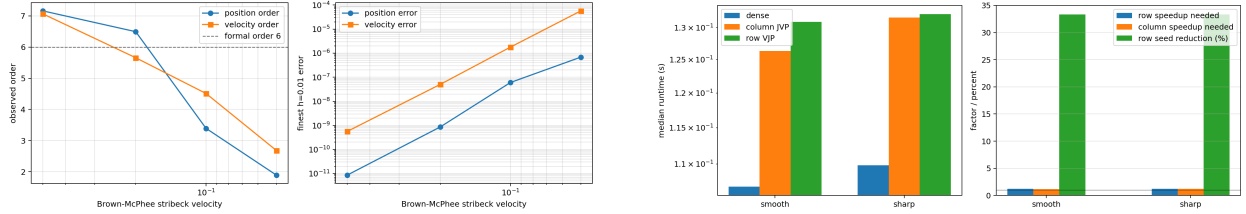


Figure 8: Two caveats that the agent must preserve: sharp-friction smoothness and sparse-backend runtime. Both are useful evidence streams, but neither supports an overbroad claim.

10.3 Open caveats

The current caveats are not secondary details; they are part of the method. Sharp-friction coarse-regime order remains practically reduced even though ultra refinement can recover high-order behavior at quantified cost. Sparse structure is correct and quantified, but the dense `jacfwd` backend remains faster in recorded wall-clock evidence. Full source-paper TFE stage replacement remains open. External source-policy superiority remains false because the full same-test benchmark campaign has not been run.

10.4 Claim boundary

Figure 9 summarizes the paper-facing boundary. The accepted claim is a narrowed conditional method claim. Diagnostic evidence remains available for future research, but not all diagnostics become claims. Open gates include source-policy external campaigns, full TFE stage replacement, and source-policy runner-package completeness.

11 Discussion

11.1 What the agent changes

The proposed agent changes the research workflow in three ways. First, it turns method identity into data. A candidate is not just a code path; it is a MethodSpec whose residual rows, endpoint map, solver policy, and theorem conditions can be audited. Second, it makes execution policy explicit. The agent decides whether a question needs a read-only validator, a bounded probe, a full generator, or an external campaign before

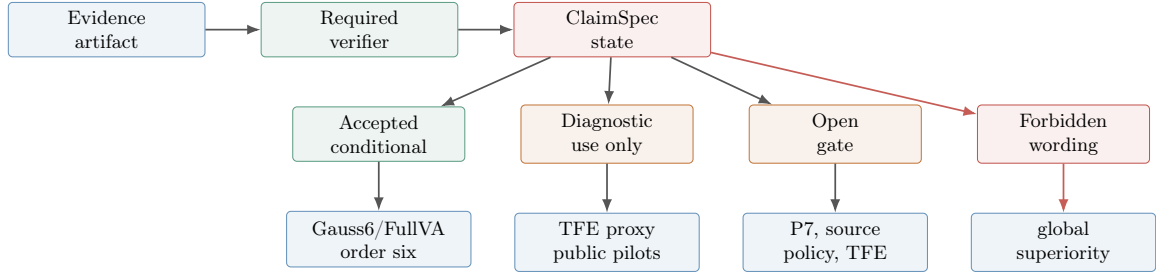


Figure 9: The claim boundary is part of the agent interface. Accepted evidence, diagnostics, and open gates are shown together so that paper prose cannot silently exceed the artifacts.

running commands. Third, it turns paper writing into a claim-state transition rather than a summarization task.

This does not remove the human researcher. The human still chooses the scientific direction: whether to prioritize a stronger TFE replacement, a better sparse backend, a closed-loop dynamic-order theorem, or an external source-policy campaign. The agent’s role is to prevent the chosen direction from corrupting the existing evidence boundary.

11.2 Why generic agents are insufficient

A generic coding agent can edit scripts and run tests, but integrator research requires a different notion of correctness. The unit of progress is not “test passed.” It is “a specific claim state changed under a specific evidence policy.” Without this discipline, an agent is likely to make the most common research mistakes: collapsing diagnostics into claims, comparing against mismatched baselines, or describing a mechanically buildable package as submission-ready.

11.3 Limitations

This paper is a methodology and case-study paper, not a controlled evaluation of agent productivity across laboratories. The architecture is grounded in a single research tree for Lie-group multibody integrators. The numerical values reported here summarize local artifacts rather than newly executed experiments. The current v047/v048 state is also intentionally incomplete: source-policy external superiority is false, full TFE stage replacement is not accepted, and full global submission readiness is not claimed. These limitations are not hidden; they are exactly the boundaries that the agent is designed to preserve.

12 Conclusion

Auto-research agents for numerical-method development should be built around verifiers, not around unconstrained autonomy. In Lie-group multibody integrator research, the agent must preserve method identity, choose sandbox levels, separate theorem inputs from diagnostics, select numerical examples by failure mode, and write claims through a state machine. The v001–v048 case study shows why this structure is necessary: each phase of the project exposed a new failure mode, and each failure mode required a new verifier. The current Gauss6/FullVA pipeline demonstrates a bounded but useful result: a conditional sixth-order local method claim with strong smooth evidence and four ASME mechanism rows, while retaining explicit caveats around full TFE replacement, external source-policy campaigns, sparse speed, and sharp-friction cost. A good agent in this setting is therefore not the one that writes the boldest paper. It is the one that keeps the paper aligned with the proof ledger and the artifacts.

Data and Artifact Availability

The manuscript draft is based on local project artifacts in the `lie_group_integrator_work` tree:

- `CURRENT_PIPELINE_CONTRACT.md`;
- `version_ledger.csv`;
- `ORDER_PROOF_LEDGER.md`;
- `paper_v047_cylindrical_chain/CURRENT_STATUS_CN.md`;
- `paper_v047_cylindrical_chain/PAPER_NUMERICAL_RESULT_MATRIX.md`;
- `paper_v047_cylindrical_chain/CROSS_PAPER_BENCHMARK_SPEC.md`;
- `paper_v047_cylindrical_chain/CMAME_PROOF_CONTRACT_GATE.md`;
- `paper_v047_cylindrical_chain/PROOF_CLAIM_TRACEABILITY_AUDIT.md`;
- `v047_cylindrical_chain_pipeline/results/summary_v047.json`;
- `v048_cross_paper_same_test_benchmarks/results/summary_v048.json`.

The numerical plots are drawn from the corresponding v047 paper figure directory; architecture, proof, ASME-lane, and claim-boundary diagrams are rendered as LaTeX/TikZ vector figures in this source. This draft revision also records its local reviewer pass in `integrator_agent_methodology_review_audit.md`.

Acknowledgments

This draft frames a methodology for coordinating lab-scale integrator research. It uses the Simulation-Based Engineering Laboratory’s versioned multibody integrator artifacts as the running case study.

References

- [1] E. Chaturvedi, C. Sandu, and A. Sandu. Higher-order integration of index-3 DAE with friction using time finite elements on Lie groups. *Multibody System Dynamics*, 2026. doi:10.1007/s11044-026-10153-w.
- [2] W. Kim and J. N. Reddy. A new family of higher-order time integration algorithms for the analysis of structural dynamics. *Journal of Applied Mechanics*, 84(7):071008, 2017. doi:10.1115/1.4036821.
- [3] E. Hairer and G. Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. Springer, Berlin, 1996.
- [4] C. W. Gear, B. Leimkuhler, and G. K. Gupta. Automatic integration of Euler–Lagrange equations with constraints. *Journal of Computational and Applied Mathematics*, 12–13:77–90, 1985. doi:10.1016/0377-0427(85)90008-1.
- [5] H. Munthe-Kaas. Runge–Kutta methods on Lie groups. *BIT Numerical Mathematics*, 38:92–111, 1998.
- [6] H. Munthe-Kaas. High order Runge–Kutta methods on manifolds. *Applied Numerical Mathematics*, 29:115–127, 1999.
- [7] O. Bruls, A. Cardona, and M. Arnold. Lie group generalized-alpha time integration of constrained flexible multibody systems. *Mechanism and Machine Theory*, 48:121–137, 2012. doi:10.1016/j.mechmachtheory.2011.07.017.
- [8] V. Wieloch and M. Arnold. BDF integrators for constrained mechanical systems on Lie groups. *Journal of Computational and Applied Mathematics*, 387:112517, 2021. doi:10.1016/j.cam.2019.112517.
- [9] L. O. Jay. Symplectic partitioned Runge–Kutta methods for constrained Hamiltonian systems. *SIAM Journal on Numerical Analysis*, 33(1):368–387, 1996. doi:10.1137/0733019.

- [10] J. E. Marsden and M. West. Discrete mechanics and variational integrators. *Acta Numerica*, 10:357–514, 2001. doi:10.1017/S096249290100006X.
- [11] S. Leyendecker, J. E. Marsden, and M. Ortiz. Variational integrators for constrained dynamical systems. *ZAMM*, 88(9):677–708, 2008. doi:10.1002/zamm.200700173.
- [12] D. E. Stewart and J. C. Trinkle. An implicit time-stepping scheme for rigid body dynamics with inelastic collisions and Coulomb friction. *International Journal for Numerical Methods in Engineering*, 39(15):2673–2691, 1996. doi:10.1002/(SICI)1097-0207(19960815)39:15<2673::AID-NME972>3.0.CO;2-I.
- [13] M. Anitescu and F. A. Potra. Formulating dynamic multi-rigid-body contact problems with friction as solvable linear complementarity problems. *Nonlinear Dynamics*, 14(3):231–247, 1997. doi:10.1023/A:1008292328909.
- [14] A. Tasora and M. Anitescu. A matrix-free cone complementarity approach for solving large-scale, nonsmooth, rigid body dynamics. *Computer Methods in Applied Mechanics and Engineering*, 200:439–453, 2011. doi:10.1016/j.cma.2010.06.030.
- [15] M. Borri and S. N. Atluri. Time-finite element method for the constrained dynamics of a rigid body. In *Computational Mechanics*. Springer, Berlin, 1988.
- [16] T. J. R. Hughes and G. M. Hulbert. Space-time finite element formulations for elastodynamics: formulations and error estimates. *Computer Methods in Applied Mechanics and Engineering*, 66:339–363, 1988.
- [17] G. M. Hulbert. Time finite element methods for structural dynamics. *International Journal for Numerical Methods in Engineering*, 33:307–331, 1992.
- [18] C. S. Jog, M. Agrawal, and A. Nandy. The time finite element as a robust general scheme for solving nonlinear dynamic equations including chaotic systems. *Applied Mathematics and Computation*, 279:43–61, 2016.
- [19] O. A. Bauchau. The finite element method in time for multibody dynamics. *Journal of Computational and Nonlinear Dynamics*, 19(7):071001, 2024. doi:10.1115/1.4063953.
- [20] P. Brown and J. McPhee. A continuous velocity-based friction model for dynamics and control with physically meaningful parameters. *Journal of Computational and Nonlinear Dynamics*, 11:054502, 2016. doi:10.1115/1.4033658.
- [21] J. Taves, A. Kissel, and D. Negrut. On an exponential map approach for rigid body kinematics and dynamics analysis. Technical Report TR-2020-08, Simulation-Based Engineering Laboratory, University of Wisconsin–Madison, 2020.
- [22] A. Kissel, D. Negrut, and J. Taves. Dwelling on the connection between $SO(3)$ and rotation matrices in rigid multibody dynamics. Part 1: Description of an index-3 DAE solution approach. In *ASME IDETC-CIE*, 2021. doi:10.1115/DETC2021-72057.
- [23] A. Kissel, D. Negrut, and J. Taves. Constrained multibody kinematics and dynamics in absolute coordinates: a discussion of three approaches to representing rigid body rotation. *Journal of Computational and Nonlinear Dynamics*, 17(10):101008, 2022. doi:10.1115/1.4055140.
- [24] L. Fang, A. Kissel, R. Zhang, and D. Negrut. On the use of half-implicit numerical integration in multibody dynamics. *Journal of Computational and Nonlinear Dynamics*, 18(1):014501, 2023. doi:10.1115/1.4056183.
- [25] A. Kissel, L. Bakke, and D. Negrut. Reducing the constrained multibody dynamics problem to the solution of a system of ordinary differential equations via velocity partitioning and Lie group integration. *Journal of Computational and Nonlinear Dynamics*, 19(7), 2024. doi:10.1115/1.4065254.
- [26] J. Wang. Current pipeline contract and v047/v048 claim-boundary artifacts. Local project artifacts in `lie_group_integrator_work`, 2026.